

Track2Data User Guide

Turn [idtracker.ai](#) output into clean, calibrated, analysis-ready tables.

© 2026 Martina Bellio. Track2Data is released under the [MIT licence](#).

Contents

- [Introduction](#): what Track2Data does, how to install it, the quick start
- [Part I: The workflow, screen by screen](#)
 1. [Project](#)
 2. [Sessions](#)
 3. [Calibration](#)
 4. [Zones \(optional\)](#)
 5. [Metadata \(optional\)](#)
 6. [Preprocessing](#)
 7. [Metrics](#)
 8. [Processing](#)
 9. [Preview](#)
 10. [Export](#)
- [Part II: Reference](#)
 - [Understanding the output files](#)
 - [Troubleshooting](#)

Introduction

Track2Data turns [idtracker.ai](#) output folders into clean, calibrated, analysis-ready tables. This guide follows the desktop app one screen at a time, using a small demo project (two animals in a circular arena). Each screenshot sits next to the explanation of the screen it shows.

The screenshots are generated from the real app by `scripts/generate_guide_screenshots.py`, so they match the current version. In the app, **Help ► Open user guide** brings you here.

Install and start

```
pip install "track2data[ui]" # or, from a clone: pip install -e ".[ui]"
track2data-gui
```

Only need the command line? `pip install track2data` gives you `track2data run project.t2d.json` (see `track2data --help`).

How the workflow is organised

The left sidebar lists the stages. A badge shows where each one stands:

Badge	Meaning
✓	complete
△	usable, but look at the tooltip (e.g. a session has no stable identities)
✗	must be fixed before you can continue
○	nothing entered yet (optional stages stay ○)

Settings save themselves. There are no Apply buttons: a change is stored a moment after you make it, and again when you leave the screen. **Next ►** stays disabled, with a tooltip saying why, until Project, Sessions and Metrics are filled in and the calibration is valid.

#	Screen	What you do
1	Project	Create or open a project
2	Sessions	Add idtracker.ai output folders
3	Calibration	Choose how pixels become body lengths or cm
4	Zones	Draw regions of interest (optional)
5	Metadata	Attach treatment, date, ... from a CSV (optional)
6	Preprocessing	Gap filling, jump removal, smoothing
7	Metrics	Pick what to compute
8	Processing	Run the pipeline
9	Preview	Check trajectories, diagnostics and metric tables
10	Export	Write CSV / Feather / Excel and load them in R or Python

Quick start (five minutes)

1. **File ► New project...**, name it. You land on *Sessions*.
2. **Add Folders...** and choose one or more idtracker.ai session folders.
3. Leave *Calibration* on **Body length** unless you know your pixels-per-cm.
4. On *Metrics*, pick a preset such as **Standard locomotor**.
5. **► Run pipeline**, then look at **Preview ► Trajectories** to confirm tracking looks right.
6. On *Export*, tick the formats you want and **Export**.

Do step 5 before trusting any number: the trajectory view is the quickest way to spot identity swaps, tracking gaps and over-aggressive smoothing.

Part I: The workflow, screen by screen

1. Project

A project is one `.t2d.json` file next to a folder of results. It stores every setting on the following screens, so you can close the app and pick up later, or re-run the identical analysis from the command line.

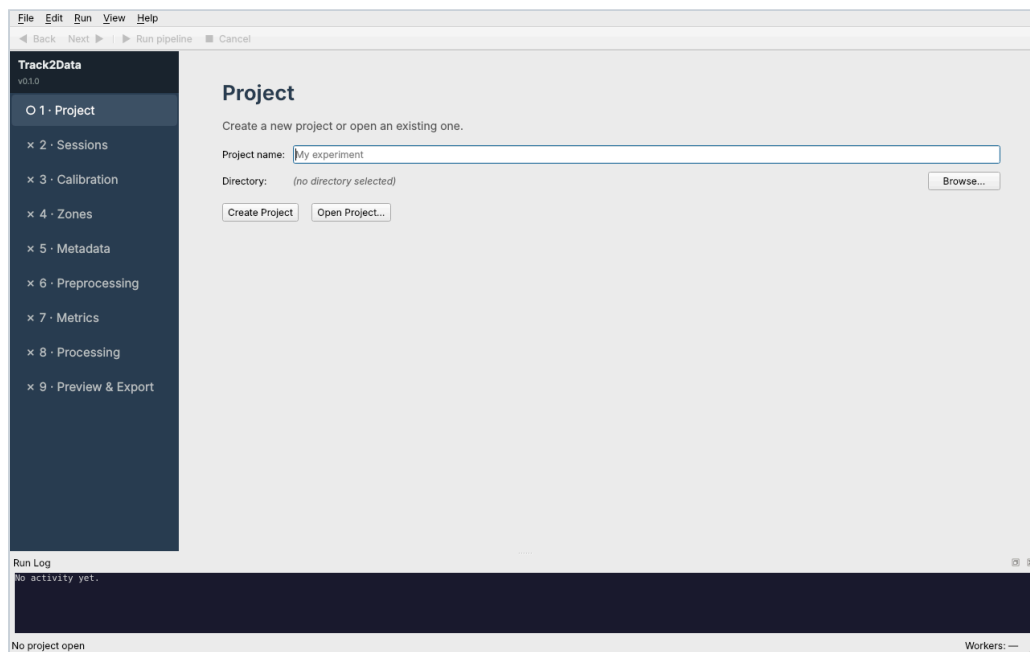


Figure 1. The Project screen, where every analysis starts.

Do this

1. **File ► New project...**, enter a name, choose a folder. Or **File ► Open project...** for an existing `.t2d.json`.
2. Track2Data moves straight on to [Sessions](#).

Notes

- **File ► Save project** writes the file; settings are also kept in memory as you edit.
- Reopening a project re-reads each session folder in the background, so frame counts and calibration readiness appear again after a moment.
- Preprocessed sessions are cached in `.t2d_cache/` inside the project folder. It is safe to delete (`track2data cache clear --cache-dir <dir>` does it too); it is rebuilt when needed.

2. Sessions

Add the idtracker.ai output folders you want to analyse (one folder per recorded video). Drag and drop works too.

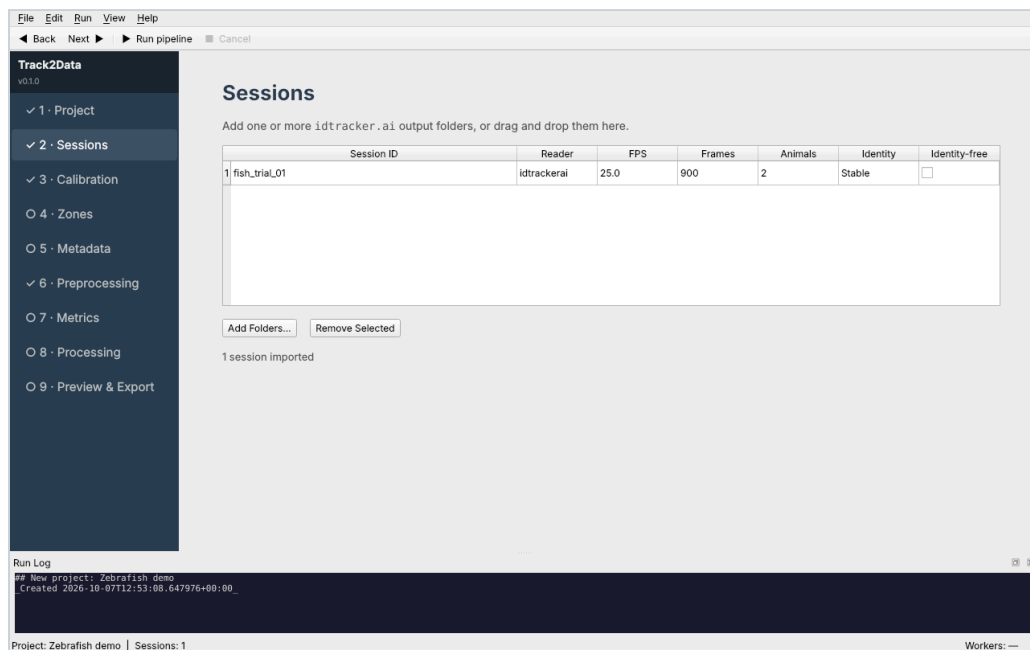


Figure 2. The Sessions screen lists every folder with what was read from it.

Each row in Figure 2 shows what was read from the folder: reader, frame rate, number of frames, number of animals, and **Identity** (`Stable` or `Unstable`).

Identity-free

Tick this if the video was tracked *without* identification, or if identities swapped so often that "animal 1" is not one animal. Sessions flagged this way (by you or by idtracker.ai's own `track_wo_identities`) skip every metric that follows an individual across frames, because those numbers would be meaningless. Group metrics and the pooled zone-occupancy metrics still run. The sidebar shows ⚠ while any session is identity-free.

Good to know

- Folders are only read, never modified.
- Adding the same folder twice is ignored.
- The status bar (bottom left) shows the session count.
- Supported: idtracker.ai 6.x output (the legacy v5 layout is also read). v4 is not supported yet (Track2Data tells you so if it recognises a v4 folder; see [sending a v4 sample](#)).

3. Calibration

Calibration decides which units distances and speeds are reported in. Figure 3 shows the default, **Body length** mode.

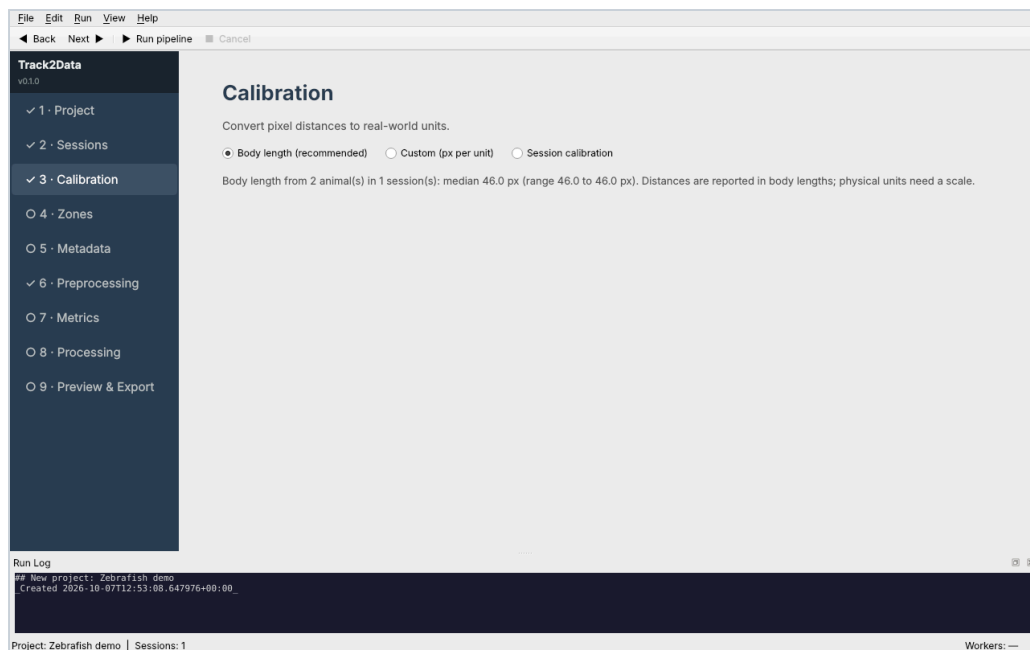


Figure 3. Calibration in Body length mode, with a summary of the body lengths read from your sessions.

Mode	Use it when	Reports
Body length (recommended)	You do not have a reliable scale	*_bl columns: distances in body lengths. Physical *_cm columns stay empty.
Custom (px per unit)	You know the scale	*_cm columns, and *_bl columns too whenever the tracker recorded body length
Session calibration	You used idtracker.ai's <i>Length Calibration</i> tool	each session's own ratio; you must confirm the unit

In Body length mode the screen summarises the body lengths read from your sessions (median and range, in pixels) so you can sanity-check them.

Custom scale: measure on the frame

Pick **Custom**, then **Measure on frame...**. Click both ends of something of known length (a ruler, the arena diameter), type its real length, and press OK. The pixels-per-unit value is filled in for you. A third click starts the measurement over.

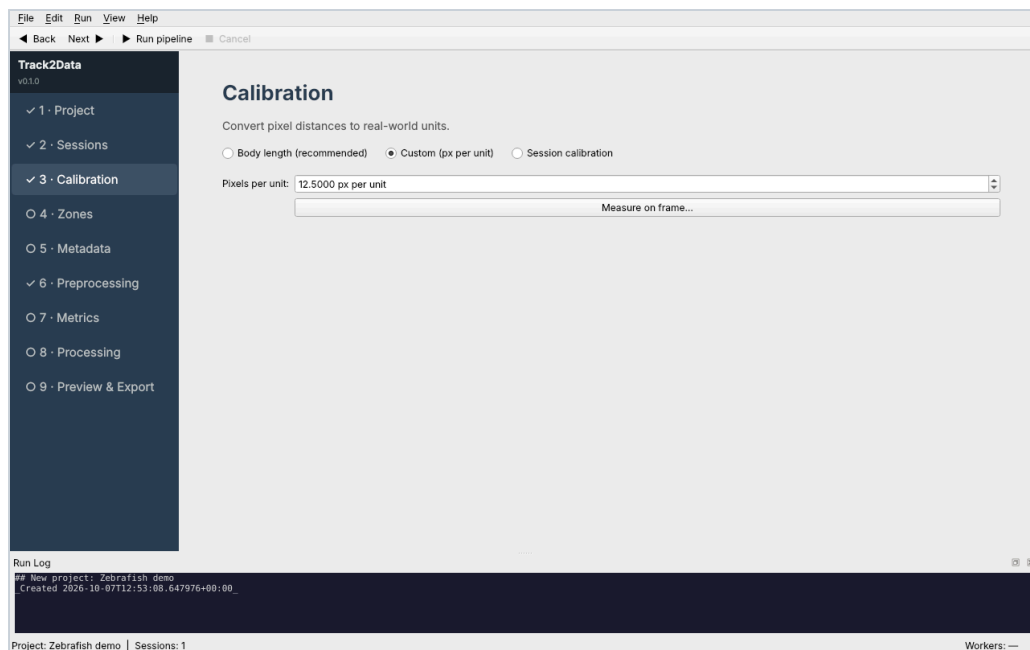


Figure 4. Custom calibration: click both ends of an object of known length to fill in the scale.

If a mode is incomplete (no scale, or an unconfirmed unit) the stage shows **X** and **Next** is disabled until you fix it.

4. Zones (optional)

Zones are regions of interest (the whole arena, a centre area, a feeder). Zone metrics need at least one. You can:

- **Import ROIs from Session:** use the polygons drawn in idtracker.ai's validator.
- **Load zones from CSV...**
- Draw your own, as described below.

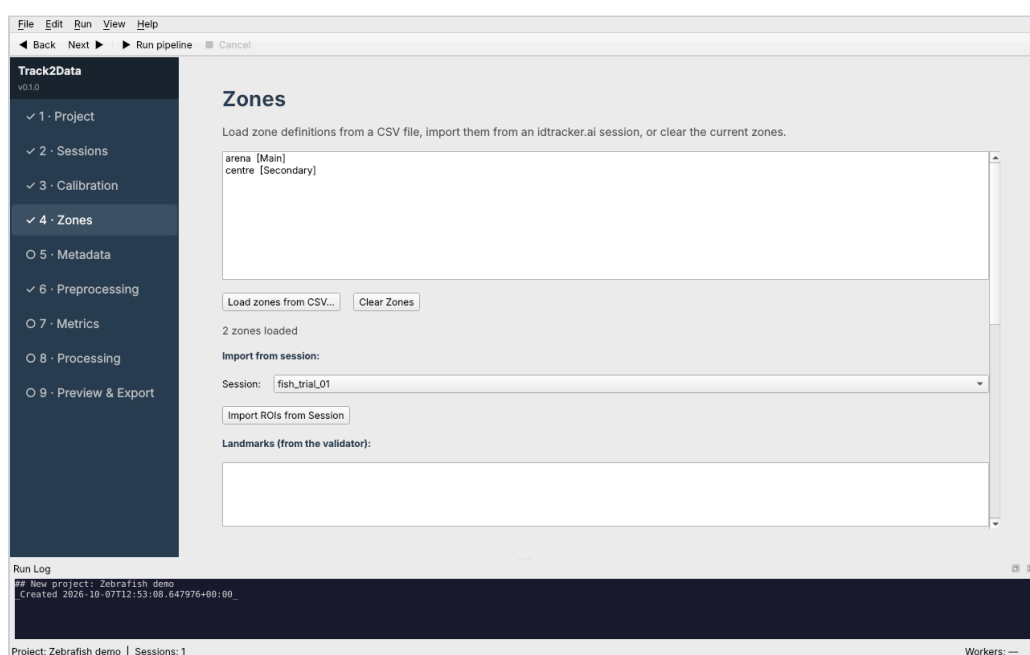


Figure 5. The zone list, with the import and load buttons.

Drawing zones

Scroll down to the canvas, which shows the session's background image. Saved zones appear shaded with their names.

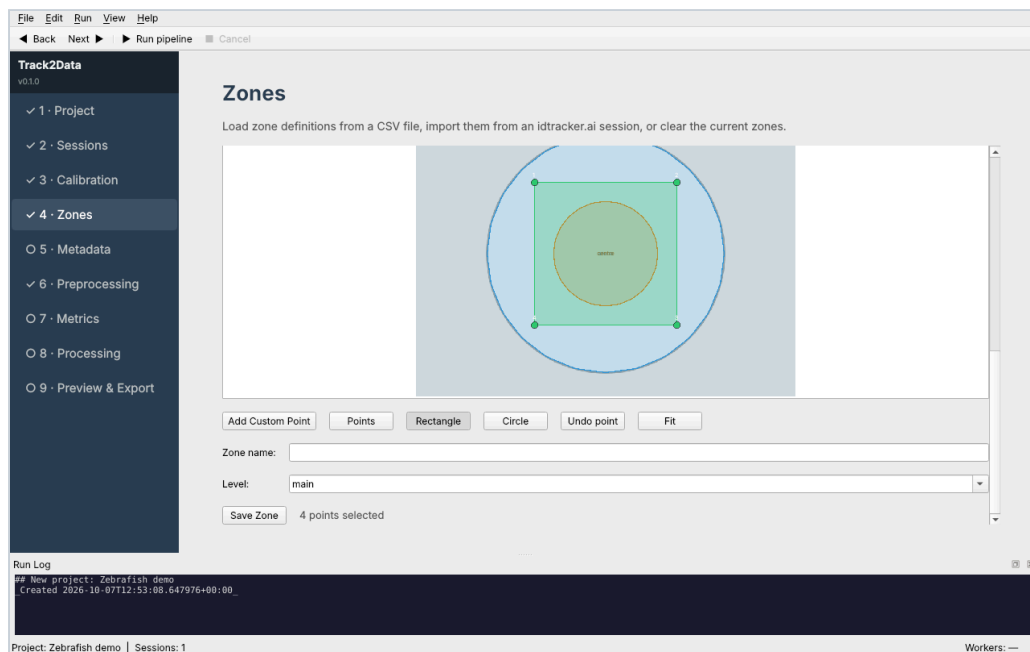


Figure 6. The zone canvas: saved zones are shaded and named, validator landmarks are blue.

Tool	How
Points	Click validator landmarks (blue) in order. Add Custom Point lets you click anywhere.
Rectangle	Drag from one corner to the opposite corner
Circle	Drag from the centre outwards
Undo point / Ctrl+Z	Remove the last vertex
Fit	Fit the whole image in view. The mouse wheel zooms; hold the middle button and drag to pan.

Custom vertices (orange/green) can be dragged to fine-tune the outline. When the shape looks right (at least 3 points), give it a **name** and a **level** (`main` or `secondary`) and press **Save Zone**.

If a zone's source resolution differs from a session's video, a yellow warning appears.

5. Metadata (optional)

Attach experimental information (treatment, date, tank, ...) to every row of the output.

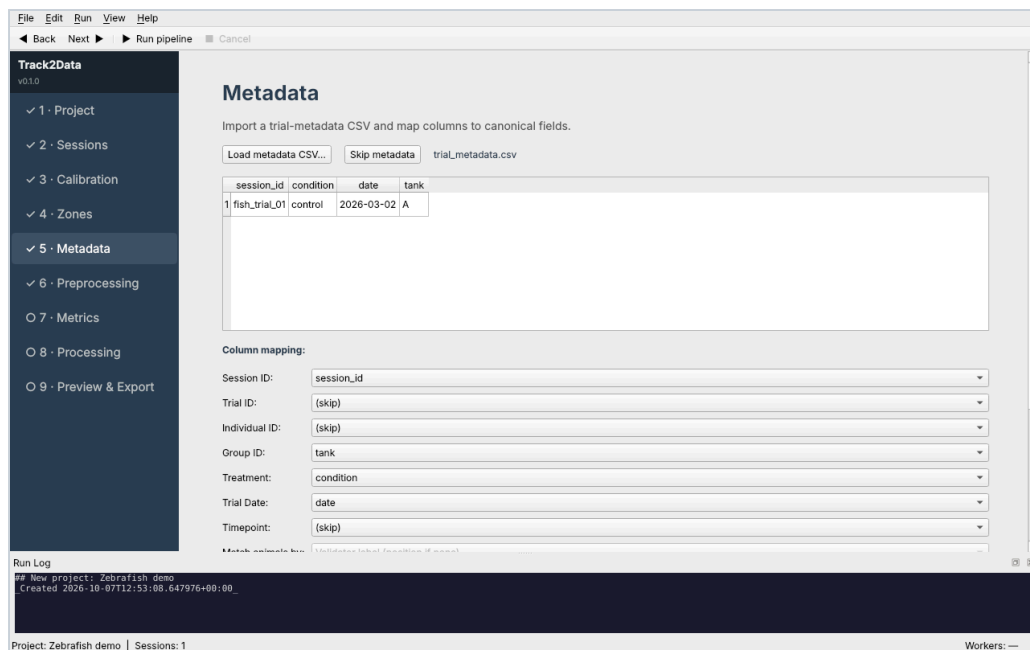


Figure 7. The Metadata screen: CSV preview, column mapping and the match summary.

1. **Load metadata CSV....** The first rows are previewed.
2. Match columns to fields. Names like `date`, `condition` or `tank` are matched automatically to *Trial date*, *Treatment*, *Group ID*; change any you disagree with, or leave a field on `(skip)`.
3. The line under the mapping tells you how many sessions found a row (`1 of 1 sessions matched`) and names any that did not.

The `session_id` column must equal the session folder name. Without an *Individual ID* column one row is matched per session (if several rows match, the first is used and the summary says so).

Per-animal information (sex, weight, genotype of individual animals)

Give the CSV **one row per animal** and map its animal column (`fish_id`, `animal_id` or any name) to *Individual ID*. Then:

- **Match animals by** decides how each value is matched to an animal. *Validator label* uses the names set in the idtracker.ai Validator (the default labels are `1`, `2`, ...) and falls back to the 0-based position for a session that has none. *Position* always uses the 0-based position (the `individual_id` of the exported tables).
- Tick the other columns you want under **Also include these columns**. Columns that are not mapped to a field and not ticked are dropped.
- Per-animal values land on every row that has an `individual_id` (the per-frame table and the individual metric tables). Values that are the same for all animals of a session (such as `treatment`) also go onto the group tables; per-animal values never do.
- The summary lists animals with no row (their cells stay empty), and CSV rows whose animal matches nothing.
- **Identity-free sessions** get no per-animal values: their rows are detection slots, not animals. Session-level fields still apply.

Pick the right match mode: if your CSV counts animals from 1 and the Validator labels are the default 1, 2, ... use *Validator label*; if it counts from 0, use *Position*. A column named like an engine column (`speed_px_s`, `frame`, `bin_index`, ...) is never carried, so it cannot overwrite data.

Skip metadata removes the file if you change your mind.

6. Preprocessing

Cleaning applied to the trajectories before any metric is computed. Defaults suit most data. Every step has an **Enabled** box; the raw data is never overwritten.

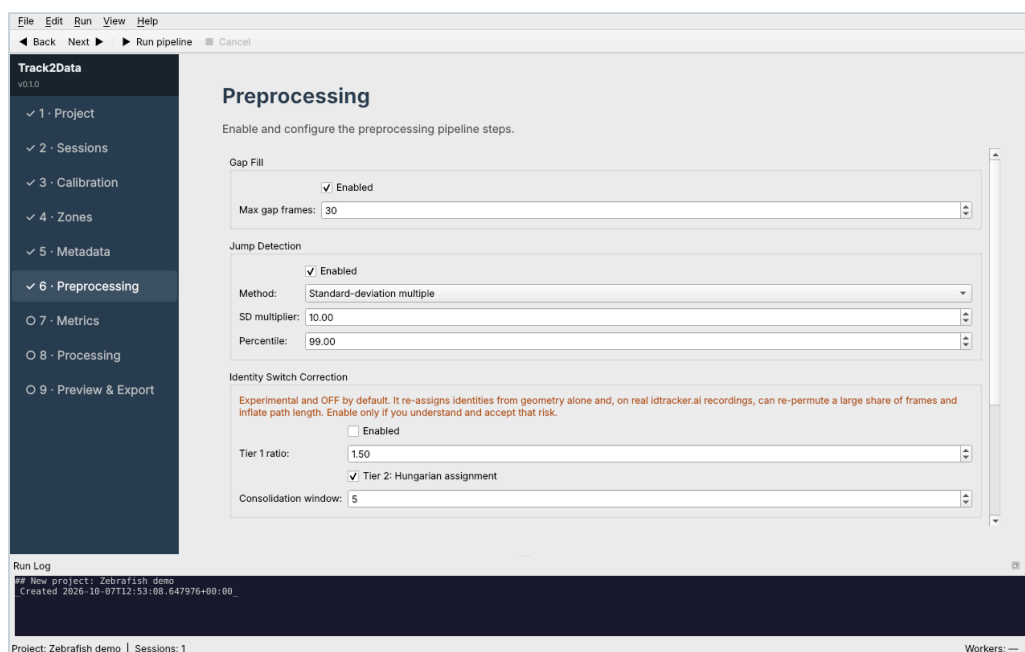


Figure 8. The Preprocessing screen with the default settings.

Step	What it does
Gap fill	Linearly interpolates tracking gaps up to <i>Max gap frames</i>
Jump detection	Finds implausible position jumps (standard-deviation multiple, percentile, or idtracker.ai's own velocity threshold) and replaces them
Identity switch correction	See below. Off by default
Smoothing	Moving average or Savitzky–Golay, over <i>Window</i> frames
Coverage gate	Warns when more than <i>Max NaN fraction</i> of an animal's frames are missing

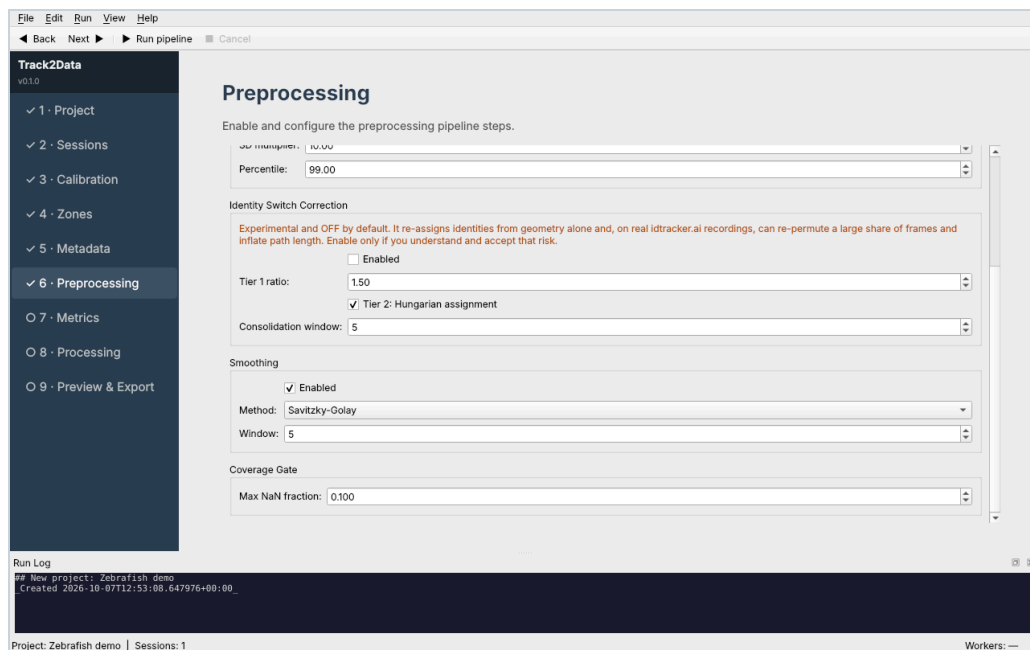


Figure 9. The identity switch correction options (experimental, off by default).

Identity switch correction is experimental and off by default. It reassigns identities from geometry alone, ignoring idtracker.ai's own fragment boundaries. On a real recording it re-permuted a large share of frames and inflated path length enormously. Leave it off unless you have checked the result in [Preview ▶ Trajectories](#) with *Raw + processed* shown.

Settings you change here are saved automatically and used for the next run.

7. Metrics

Choose what to compute. Metrics are grouped on three tabs: **Individual** (per animal), **Group** (across animals) and **Zone** (needs zones; the tab is disabled without them).

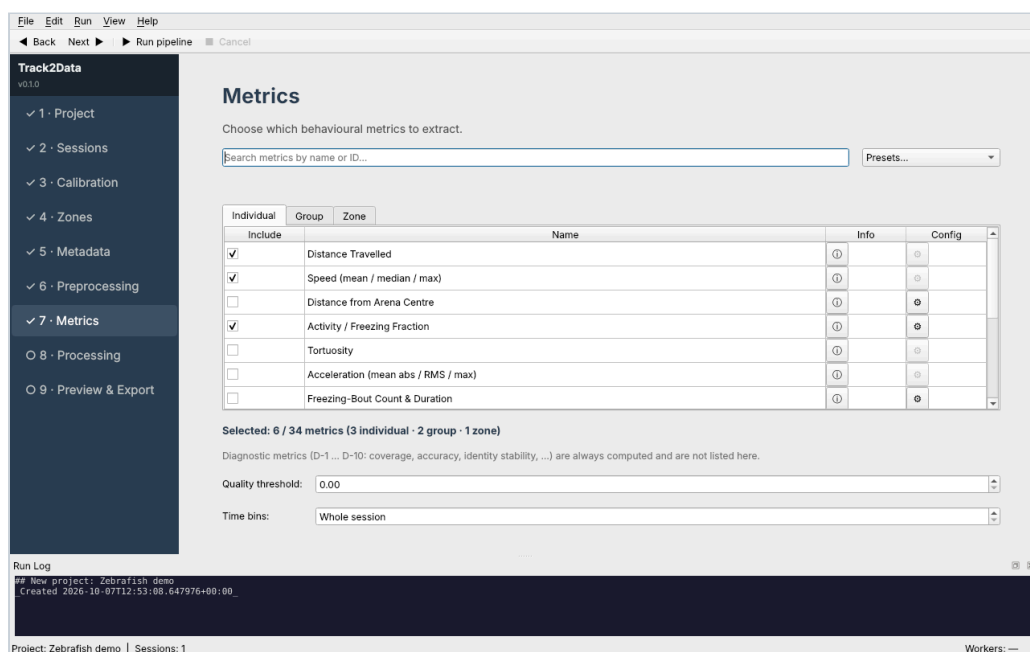


Figure 10. The Metrics screen, with search, presets and the three metric tabs.

- **Search** filters all tabs by name or ID (`speed` , `IL-2`). The line beneath says which tabs match.
- **Presets** replace the current selection: *Standard locomotor*, *Thigmotaxis & space use*, *Social dynamics*, *All metrics*.
- ⓘ opens the definition, formula and literature reference of a metric; ⚙ edits its parameters (available on metrics that have any).
- The counter shows how many are selected.
- **Quality threshold** drops frames whose identification probability is below the value.
- **Time bins** splits every session into bins of the chosen length (minutes) and reports each metric per bin, with `bin_index`, `bin_start_s` and `bin_end_s` columns. *Whole session* (the default) turns it off. Whole-track metrics (tortuosity, home-base stability) stay one row per animal with empty bin columns; diagnostics are always whole-session. Thresholds such as the freezing speed threshold are fixed from the whole session, so bins are comparable.
- Rows are greyed out for sessions without stable identities; see [Sessions](#).

Diagnostics (coverage, tracking accuracy, identity stability, ...) are always computed and are not listed here; they appear in [Preview ▶ Diagnostics](#).

Every metric is documented in [docs/METRICS_SPEC.md](#).

8. Processing

Validate pipeline checks the project and lists anything that would stop a run. **Run pipeline** (or the toolbar button, or Ctrl+R) processes every session: import, preprocessing, calibration, zone assignment, metrics, export.

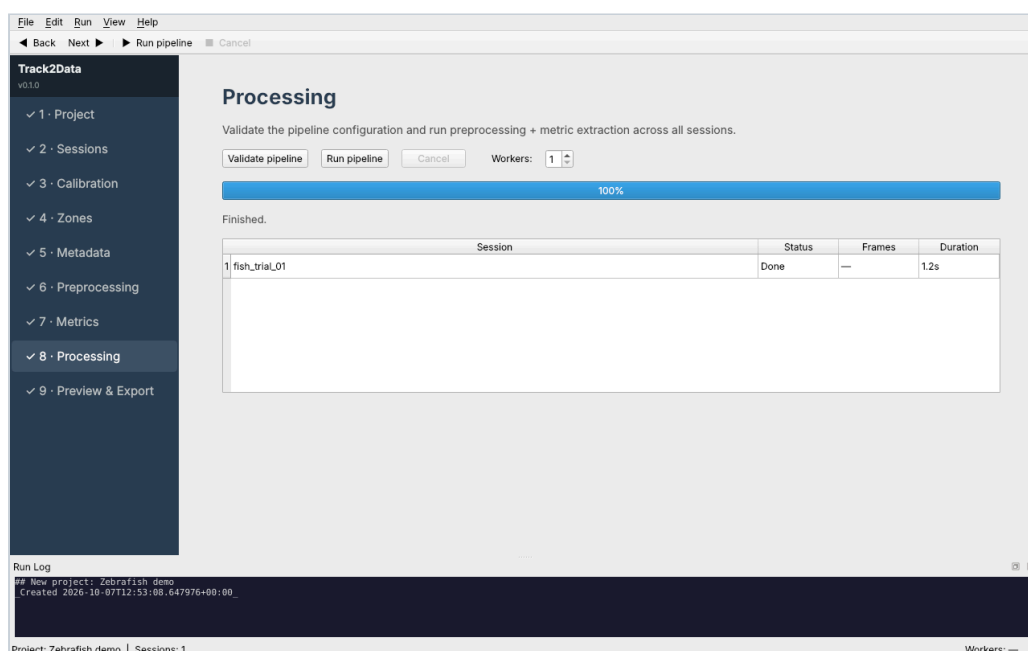


Figure 11. The Processing screen: one row per session, with status and duration.

- The table shows each session's status and duration. A failed session is marked *Failed* and does not stop the others.

- **Workers** sets how many sessions run at the same time. 1 runs them one after another. More workers help when each session takes many seconds or more; for short sessions the start-up cost of each worker makes it slower.
- **Cancel** stops the run at the next safe point (between sessions, preprocessing steps or metrics).
- Preprocessed sessions are cached, so re-running after changing only metrics or export settings skips the slow parts.

Results are written to `<project>/exports/<timestamp>/<session>/`.

9. Preview

Look before you export. Four tabs: **Summary, Diagnostics, Metrics, Trajectories**.

Trajectories

Choose a session and **Load trajectories** (it reuses the cache after a run). Then:

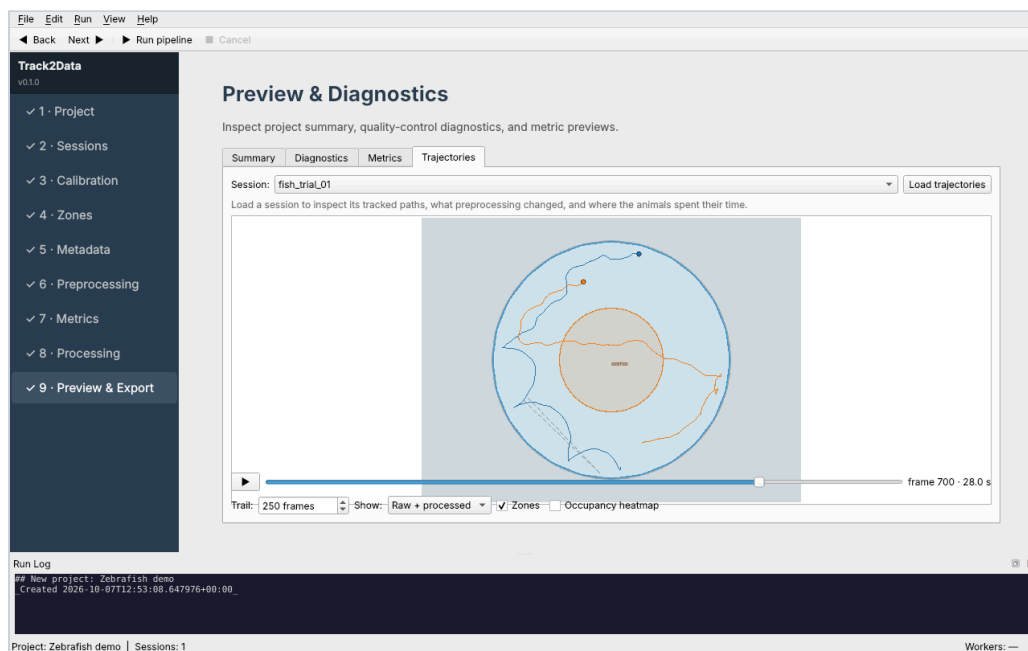


Figure 12. Trajectories with Raw + processed shown: the dashed grey line is a tracking jump that was removed.

- Drag the slider, or press ►, to move through time. **Trail** sets how many past frames are drawn.
- **Show ► Raw + processed** draws the raw path as a dashed grey line next to the processed one, so you can see exactly what gap filling, jump removal and smoothing changed. In Figure 12 the dashed line is a tracking jump that was removed.
- **Zones** shows or hides the saved zones. The mouse wheel zooms.

What to look for: a path that suddenly jumps across the arena (identity swap or detection error); long straight lines where an animal was lost; smoothing that cuts corners of real turns.

Occupancy heatmap

Tick **Occupancy heatmap** to see where the animals spent their time (red = most).

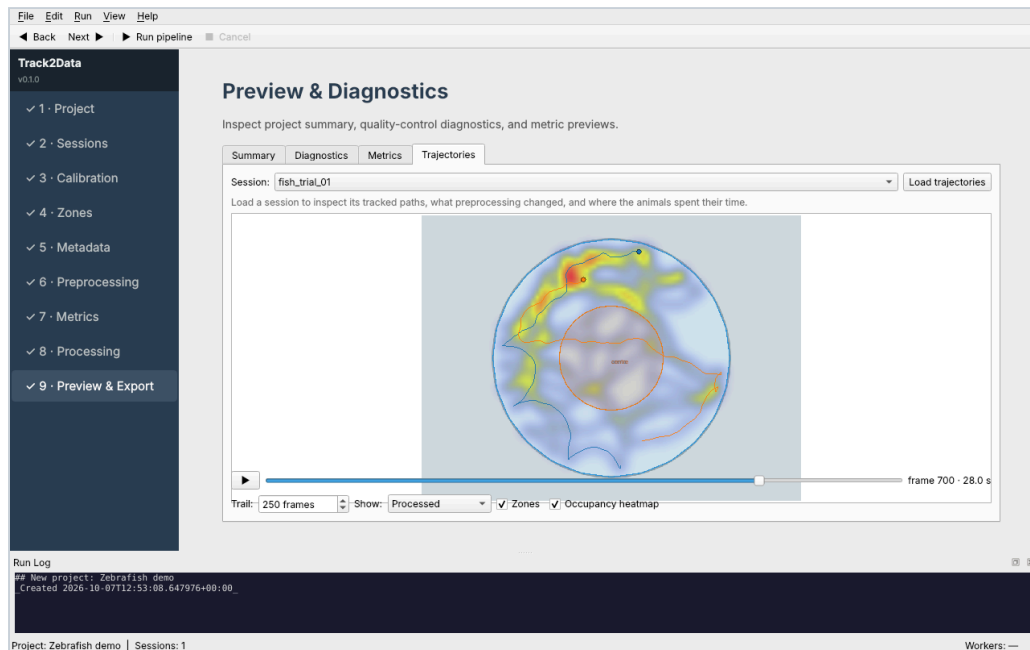


Figure 13. The occupancy heatmap.

Diagnostics

Per-animal tracking coverage and identification probability, session-level accuracy, inconsistent frames and identity stability, plus the list of preprocessing steps and how many frames each one changed.

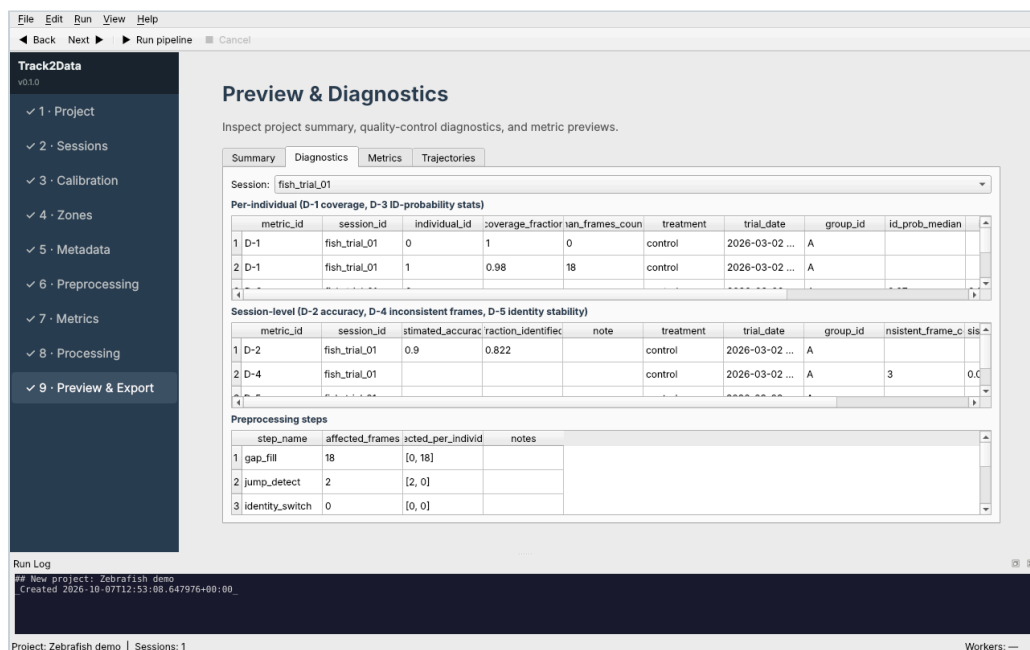


Figure 14. The Diagnostics tab.

Metrics

A table preview of each selected metric for the chosen session.

10. Export

Tick the formats you want, choose where to write them, and press **Export**.

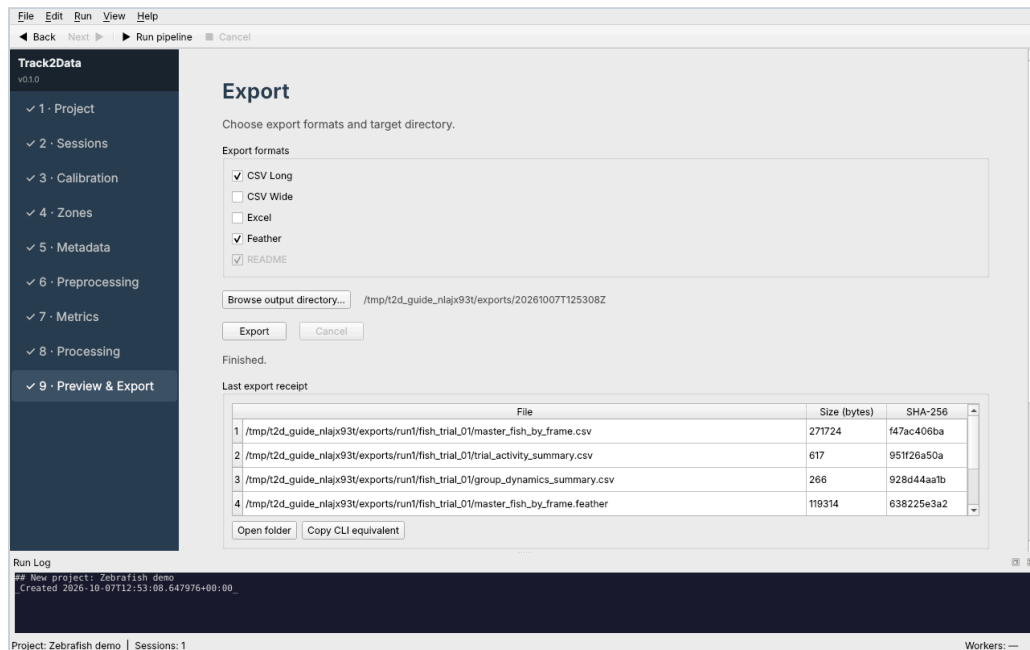


Figure 15. The Export screen, with the file receipt after an export.

1. Tick the formats you want.

Format	Files
CSV Long	master_fish_by_frame.csv , trial_activity_summary.csv , group_dynamics_summary.csv
CSV Wide	trial_summary_wide.csv
Excel	Track2Data_<project>.xlsx (one sheet per table)
Feather	master_fish_by_frame.feather , trial_activity_summary.feather
README	provenance record (always written alongside the others)

2. **Browse output directory...** or keep the default.
3. **Export.** Metrics and files are produced again, but preprocessing comes from the cache when nothing relevant changed.

The **receipt** lists every file with its size and SHA-256. **Copy CLI equivalent** gives the `track2data run ...` command that reproduces this export.

Very long recordings: Excel limits a sheet to about one million rows, so the per-frame table continues on *Fish by Frame 2*, ... The CSV and Feather files always hold the whole table.

Load the data in R or Python

After an export, the panel at the bottom gives ready-to-paste code for the files just written (R/tidyverse or Python/pandas). **Copy code**, paste, run.

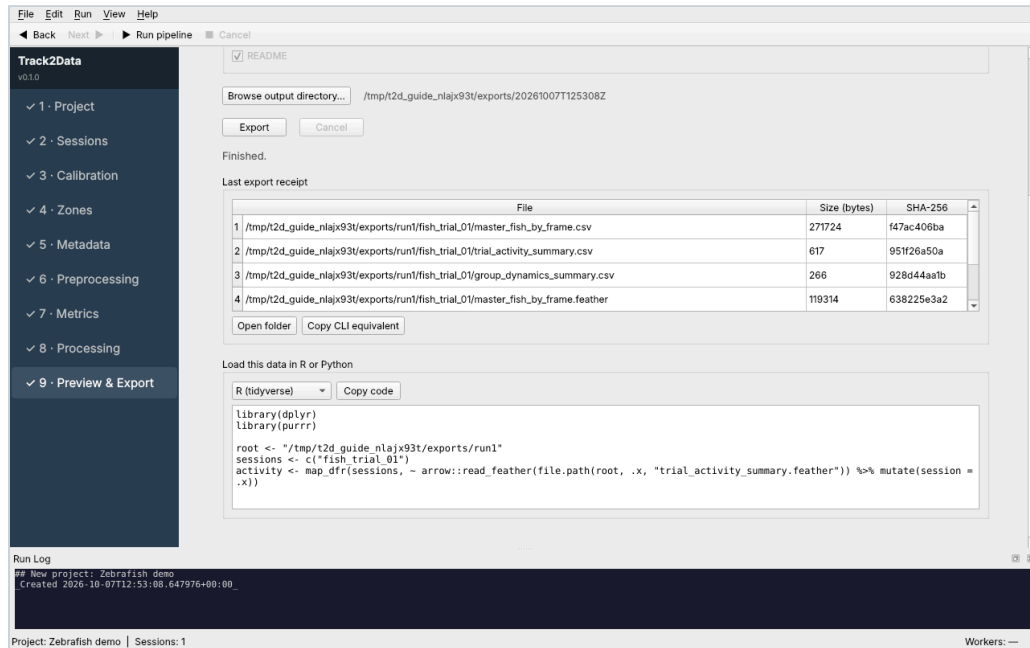


Figure 16. Ready-to-paste R and Python snippets for the files just written.

See [Understanding the output files](#) for what the columns mean.

Part II: Reference

Understanding the output files

Each session is written to `<out_dir>/<session_id>/`. Files differ by exporter; the tables are the same.

Table	One row per	Contents
<code>master_fish_by_frame</code>	session × animal × frame	<code>time_s</code> , <code>x_px</code> , <code>y_px</code> , <code>was_interpolated</code> (true where preprocessing filled a gap), <code>speed_px_s</code> , <code>heading_rad</code> , <code>main_zone</code> , <code>sec_zone</code> , plus calibrated and metadata columns
<code>trial_activity_summary</code>	session × animal	individual-level metrics, plus metadata
<code>group_dynamics_summary</code>	session	group-level metrics, plus metadata
<code>trial_summary_wide</code>	session × animal	all summary metrics side by side

- **Time bins** (when set on the Metrics screen): summary tables have one row per animal *per bin*, with `bin_index`, `bin_start_s`, `bin_end_s`; `master_fish_by_frame` gets `bin_index`. Metrics that are not meaningful per window keep a single row with empty bin columns.

Column naming

Suffix	Meaning
<code>_px</code>	pixels
<code>_cm</code>	centimetres (only with a scale; otherwise empty)
<code>_bl</code>	body lengths (needs the tracker's body length; independent of the calibration mode and of any physical scale)
<code>_s</code>	per second

Things to know

- **Empty `*_cm` columns** mean no pixels-per-unit scale was set, not an error. Use `*_bl`, or set a scale on the [Calibration](#) screen.
- **Identity-free sessions** have no per-animal rows for metrics that follow an individual; zone occupancy (Z-1, Z-2, Z-8) is reported pooled over animals, without an `individual_id` column.
- **Missing positions** are empty values, never zeros.
- The README written next to the files records the settings and versions used, and the SHA-256 of each output, so a result can be traced and reproduced.

Definitions, formulas and references for every metric: [docs/METRICS_SPEC.md](#).

Troubleshooting

You see	Why	What to do
Next ▶ is disabled	A required stage is empty or invalid	Hover over Next: the tooltip names the problem. Check the X / ○ badges in the sidebar
"No reader recognised the session folder"	The folder is not an idtracker.ai output	Pick the session folder itself (the one containing trajectories/). Supported: idtracker.ai 6.x output (the legacy v5 layout also works); v4 is not supported yet (a v4-looking folder gets a specific message; see sending a v4 sample)
Session frames / animals show –	The folder is still being read	Wait a moment; a failed read is reported in the Run Log
*_cm columns are empty	No pixels-per-unit scale	Use *_bl columns or set a scale in Calibration
△ on <i>Sessions</i>	A session is identity-free	Expected for sessions tracked without identities; see Sessions
Individual metrics missing for a session	That session is identity-free	Untick <i>Identity-free</i> only if identities really are stable
"N of M sessions matched" with names listed	Those session_id values are not in the metadata CSV	Fix the CSV or the session folder names
A zone warning about resolution	Zones were drawn on a different image size	Redraw them on this session, or confirm the sizes really match
Path length looks far too large	Jumps or identity swaps	Open Preview ▶ Trajectories with <i>Raw + processed</i> ; keep <i>Identity switch correction</i> off
Excel file has <i>Fish by Frame 2</i>	The table exceeded Excel's row limit	Use the CSV or Feather file for the complete table
Parallel run slower than sequential	Short sessions; each worker takes time to start	Use 1 worker
macOS says the app is damaged / Windows SmartScreen blocks it	Releases are not signed yet	See docs/CODE_SIGNING.md

Still stuck? Open the **Run Log** (View ▶ Toggle Run Log), then an issue on GitHub with its text.